What Should the Scheduler Know?

Compiler-Derived Behavioral Hints in a From-Scratch x86-64 Operating System

Teye John Harrison Junior*

Technical report, November 1, 2026

Abstract

A CPU scheduler must guess how each program will behave, so it infers that behavior from how the program has run so far. A compiler sees the program *before* it runs. This work asks whether static knowledge from a compiler, delivered to the kernel as a declared behavioral hint, can replace or improve runtime inference, and when it should not be trusted. I built the complete stack from scratch: a preemptive x86-64 kernel in Rust; a native code generator for PlainLang, an English-like language, differentially tested on 232 programs inside the kernel; a static classifier that labels programs CPU-bound, I/O-bound or interactive; and a deterministic benchmark harness under instruction-counted emulation. Across more than 3,400 runs and six policies, hint-driven scheduling reduced interactive wake-up latency from 7–10 ms (round-robin, MLFQ) to about 2 µs. However, a CFS-style fair scheduler with *no* hints matched it. Hints helped measurably with efficiency, cutting context switches by up to 14× at unchanged completion time. Used as priorities, wrong hints were costly (106 ms); a hybrid that uses hints only to size time slices held about 3 µs even with every hint wrong. The classifier matched measured behavior on 76% of 150 random programs, and every error underestimated demand. In the kernel, the direction of an error determined who paid for it. Predictions were committed before each experiment, and failures are reported alongside successes.

1 Introduction

Operating-system schedulers decide, many times per second, which program runs next. They usually cannot know what a program will do, so they infer it from history. Multi-level feedback queues (MLFQ) demote threads that use their whole time slice and favor those that block early [1, 2]. Fair schedulers such as Linux's Completely Fair Scheduler (CFS) track how much CPU time each thread has received and run the least-served one [5]. Both learn from behavior after the fact.

A compiler has a different vantage point. It sees loops, their bounds, and the places where a program waits, before the program has executed a single instruction. That suggests a question that is easy to state and harder to answer honestly:

> *Can compiler-derived information about a program's behavior make CPU scheduling better, and under what conditions does it make it worse?*

Answering this needs control over every layer at once: the language, the compiler, the program format that carries the hint, the kernel that schedules, and the instrumentation that measures. No off-the-shelf stack gives that, so I built one.

Contributions.

1. A full experimental platform: a from-scratch x86-64 kernel with six switchable scheduling policies and microsecond-resolution per-thread accounting, a compiler from PlainLang to native x86-64, a static behavior classifier, and an unattended, deterministic benchmark harness (§2).
2. Evidence that hints beat textbook schedulers by three to four orders of magnitude on interactive latency, but that a hint-free fair scheduler matches them; the measurable value of hints is fewer context switches (§4).
3. A hybrid design, `hfair`, in which hints influence only time-slice length. It keeps fair scheduling's latency, recovers most of the switch savings, and cannot be harmed by a wrong hint.
4. An accuracy study of the compiler's classifier on 150 random programs, showing a one-directional error bias, and a kernel experiment showing who bears the cost of each kind of error (§5).
5. A pre-registered methodology in which predictions were written and committed before each experiment, with every failed prediction reported (§3).

2 System

2.1 The kernel

AetherOS is written in Rust without the standard library and boots on x86-64. It provides per-process page tables, ring-3 user programs, system calls through

Policy	Decision rule
`rr`	One queue, fixed quantum Q.
`mlfq`	3 levels, slice $Q \cdot 2^{\ell}$; demote after using a level's allotment; periodic boost to the top (200 ms).
`hint`	Priority from the declared hint (interactive > io > cpu); threads waiting > 100 ms are aged to the top.
`hmlfq`	MLFQ whose *starting* level comes from the hint.
`fair`	CFS-style: run the thread with least virtual runtime; slice $\max(Q/n, 1\,\text{ms})$; sleeper credit; wake-up preemption. No hints.
`hfair`	`fair`, but a thread hinted `cpu` gets a slice $k\times$ longer (default $k = 4$). Hints never affect *who* runs.

Table 1: Scheduling policies.

both `int 0x80` and `SYSCALL/SYSRET`, NX, SMEP and SMAP protection, a small on-disk file system, and preemptive scheduling driven by a 1 kHz timer. Each context switch saves and restores the SSE register file, verified with a negative control. Blocking uses real wait queues rather than polling, and an idle thread never consumes a time slice. The scheduler records, for every thread, its response time (creation to first dispatch), turnaround, CPU time, time spent ready but not running, and the latency of every single wake-up (woken to actually running).

2.2 Scheduling policies

All six policies are switchable at run time (Table 1). The base quantum is $Q = 10$ ms unless stated.

2.3 From source code to hint

PlainLang is an English-like teaching language (`repeat 30 times`, `wait 0.004`). I wrote a backend that emits x86-64 machine code directly into the kernel's program format, with the hint embedded in the file header. Its correctness was established by differential testing: 232 programs, including 200 randomly generated ones, produce identical output inside the kernel and in the reference interpreter.

The classifier estimates a program's *work* (operations, multiplied through loop trip counts) and its number of *waits*, and labels it by work per wait: interactive below 2,000 operations, I/O-bound below 100,000, CPU-bound above. Its *counted* analysis adds constant propagation and a conservative recognizer for counted `while` loops. Loops whose length it cannot determine are assumed to run 100 times. Every hint comes with a written rationale, so a wrong guess can be traced to the rule that produced it.

2.4 Measurement

A host-side harness writes a plan of benchmark lines onto a virtual disk, boots the kernel unattended in QEMU, and parses its serial output. QEMU runs with `-icount shift=3,sleep=off`, so guest time advances 8 ns per executed instruction, independent of host load. Two boots with identical inputs agreed within 0.1%. Workloads are built from three synthetic programs: `wcpu` (pure computation), `wio` (30 rounds of ∼4.8 ms work then a 4 ms wait) and `wint` (30 rounds of a 15 ms wait then ∼0.5 ms work), plus PlainLang programs whose hints come from the compiler. Statistics are means with 95% confidence intervals over seeds (10 for main experiments, 3–5 for sweeps). The primary dataset uses the optimized kernel build. An earlier unoptimized build overstated scheduling costs by 10× and is not used for any claim.

3 Methodology

For every experiment after the first round, predictions were written into the plan file's header and committed to version control *before* the experiment ran. They were then scored by scripts rather than by eye. Where an explanation was developed after seeing a failure, it was treated as a hypothesis and given its own pre-registered test. Each measurement problem found along the way is documented, and affected data was discarded and rerun. These included a starved benchmark launcher, an $O(n)$ frame allocator that made the first run of each boot slow, a parser that dropped one row in 24, and a boost timer that carried state between runs (§6).

4 Results

4.1 Against textbook schedulers

Correct hints cut interactive latency by three to four orders of magnitude. On a workload of two CPU-bound processes, one I/O-style and one interactive, the interactive process's p95 wake-up latency is 2 µs under `hint`, against 9.8 ms under `rr` and 6.8 ms under `mlfq`. A woken interactive thread has strictly higher priority and preempts within one timer tick.

The MLFQ result depends on a constant I chose. Sweeping the boost period showed that MLFQ's periodic boost *hurts* interactive latency, the opposite of my prediction. The boost returns every CPU-bound thread to the interactive thread's level. Without the boost, MLFQ reaches 1–2 µs, but the I/O-style process sinks to 76 ms p95. A comparison against a single MLFQ configuration would have been misleading.

Wrong hints are worse than none. With deliberately inverted hints, `hint` gives the interactive process 106 ms p95 latency, bounded only by the 100 ms aging threshold. Across threshold settings, latency tracked the threshold (27, 67, 106, 206 ms). With aging disabled, the victim was *starved*: its first dispatch came after 1.29 s. Its wake-up latency still looked small (p95 0.8 ms), because wake-up latency cannot see a thread

Workload	`rr`	`fair`	`hint`	$\texttt{hfair}_{16}$
4 CPU-bound	268	1,295	86	92
mixed (`all`)	201	419	153	157
6 CPU + 1 interactive	437	3,938	248	293

Table 2: Context switches per run (means).

Analysis	Agreement	Too light	Too heavy
basic	62%	56	1
counted	**76%**	36	0
safe (heavy-biased)	59%	11	50

Table 3: Classifier vs. measured behavior, 150 programs. "Too light" means the guessed class demands less CPU than measured.

that never ran. All reports now include response time and a starvation audit.

4.2 Against a fair scheduler

A hint-free fair scheduler matches hints on interactive latency. `fair` achieves 3 µs against `hint`'s 2 µs. With many competitors, `fair` is better at the default aging setting (5 µs vs. 3.6 ms with ten CPU-bound processes). A sweep showed the hint scheduler's degradation is caused entirely by its aging rule, which lifts a waiting CPU-bound thread straight to the interactive level. Replacing it with graded aging (one level per 100 ms waited) removed that tail (3.6 ms to 4 µs) and two other anomalies traced to the same rule. The cost: a victim of wrong hints waits about twice as long before rescue (206 ms instead of 106 ms), though it is never starved. The mechanism behind `fair`'s result is simple: a thread that sleeps has used little CPU, so its virtual runtime is small and it is picked first. The scheduler measures what the hint declares. `fair` also handles programs that break the hint scheduler. For a program interactive in one phase and CPU-bound in the next, `hint` gives 65 ms latency and `fair` 0.002 ms.

What hints still buy is fewer context switches. Makespan is within about 1% for every policy, so total throughput does not change. Switch counts differ greatly (Table 2). Because the emulator has no caches or TLB, real switches cost more than measured here, which makes this the result most worth carrying to real hardware. Lower mean turnaround for CPU-bound work under `hint` is an ordering effect, not a throughput gain. One qualification matters: `fair`'s switch count is itself set by its minimum slice. Raising that floor from 1 to 5 ms cut switches to 0.21× with interactive latency unchanged (3–6 µs). Hints therefore buy switch savings *without* lengthening every slice, not savings that are unavailable any other way. The same holds inside `hint`: with equal slices at every level, its switch count on four CPU-bound programs rose from 85 to 267, matching round robin (268). The entire switch advantage comes from *which* threads receive long slices, not from priority. That is precisely the part `hfair` retains.

4.3 A hybrid: hints size slices, measurement picks who runs

In `hfair`, a `cpu` hint only lengthens that thread's slice. Selection is still by virtual runtime, so a hint cannot delay anyone else by more than the slice factor. With multiplier 16, switch counts come within 3–18% of `hint`'s while interactive latency stays at 3–4 µs. With *wrong* hints, interactive latency stays at 3 µs (`hint`: 106 ms) and switch counts fall back to `fair`'s level. A wrong hint removes the optimization and nothing else. On PlainLang programs with compiler-derived hints, `hfair` gives 0.003 ms interactive latency with about 45% fewer switches than `fair`.

4.4 Out of sample: random workloads

Because `hfair` was designed after seeing the data it explains, I tested it on 30 randomly drawn workloads (1–8 CPU-bound, 0–3 I/O-style, 0–3 interactive processes; 720 runs per batch) with predictions committed first. The structural claims replicated: the ordering switches(`hint`) ≤ switches(`hfair`) ≤ 1.05 · switches(`fair`) held in all 30 mixes in both batches, and wrong hints broke `hint` in every mix with both CPU-bound and interactive processes (up to 164 ms response time). Makespan stayed within 1.14%. Three latency predictions failed. Every failure involved several interactive processes, and the explanation I proposed afterwards was given its own test. Wake-ups that fall in the same 1 ms timer tick must queue behind each other whatever the policy. With no competing processes at all, two interactive programs had 3.3% of wake-ups delayed and three had 5.6–5.8%, identically under all five policies. Turning off aging restored `hint`'s latency to 3–4 µs in every mix with at most two interactive processes. All four predictions of that follow-up held. The lesson: p95 latency is a fair summary only with one interactive process, because with several there is a floor that belongs to the workload, not to any scheduler.

5 How good is the compiler?

5.1 Accuracy on random programs

Five hand-written programs say little about a classifier, so I generated 150 PlainLang programs from a fixed seed. They mix counted and uncounted loops, loop bounds hidden behind function calls, counters aliased through other variables, waits inside conditionals, and two-phase programs. Each was run *alone* in the kernel, and its measured CPU time per wait served as ground truth (Table 3).

The counted analysis was right 97% of the time on counted `while` loops, and only 53–67% on its documented blind spots. Just 2 of 150 were gross errors (CPU-bound labeled interactive or vice versa). The finding I did not predict: **all 36 errors went the**

same way. Every wrong guess underestimated demand. Two rules cause this. Unknown loops are assumed to run 100 times, and a wait inside a conditional is counted as if it happened on every iteration.

Biasing the classifier toward heavy guesses (unknown loops assumed to run 10,000 times, a conditional counting its fewest waits) did not fix this. It moved the errors to the other direction and cut accuracy to 59%. Three of its four pre-registered predictions failed. When loop bounds span four orders of magnitude, no constant guess for an unknown bound is right.

5.2 Who pays for a wrong hint?

I ran ten mislabeled programs from the corpus next to a real interactive program and a CPU hog (120 runs, predictions committed first). A *too-heavy* hint cost only the mislabeled program: under `hint` its own p95 latency rose from about 1 µs to exactly 100 ms, the aging threshold, while its neighbor stayed at 1 µs in every case. A *too-light* hint delayed the neighbor by more than 1 ms in only 2 of 5 cases, and the harm scaled with the program's burst length. Under `hfair`, changing the hint changed the neighbor's latency in none of the cases beyond the tick-collision floor. So the classifier's characteristic error is the dangerous kind for hint-as-priority, and a harmless kind for `hfair`.

6 Threats to validity

- **Emulated hardware.** Every instruction costs the same; there are no caches, pipelines or TLBs. Only relative comparisons are meaningful.
- **Synthetic workloads.** "I/O" is a timed sleep and "user events" arrive on fixed periods. The classifier corpus is generated, not written by real users.
- **An approximation of CFS.** `fair` has no weights, nice levels, groups or EEVDF deadlines [4], and runs on one CPU.
- **The author wrote the baselines.** Constant sweeps (boost, aging, granularity, sleeper credit, MLFQ depth, slice floor) showed some conclusions depend on such constants: MLFQ's interactive latency ranged from 4.4 to 18.8 ms with its number of levels alone.
- **Measurement apparatus.** A seed-for-seed regression check found that MLFQ's boost timer carried state between runs, making MLFQ-family results depend on plan order (up to 29 ms in one metric). It was fixed and the entire dataset rerun on one kernel image.
- **Post-hoc elements.** The `hfair` design and the tick-collision explanation came after looking at data; both received separate pre-registered tests.

7 Related work

MLFQ originates with CTSS [1] and is presented in standard texts [2]. Proportional-share scheduling includes lottery and stride scheduling [3] and virtual-time schedulers such as EEVDF [4], which Linux adopted after CFS [5]. A systematic review of work on passing program-level hints to schedulers is left to the author's ongoing literature review. This report makes no claim of novelty until that review is complete.

8 Conclusion

Compiler hints look transformative against textbook schedulers, and that is the wrong comparison. Against a scheduler that measures CPU usage directly, declared behavior adds little to latency, because the scheduler already sees it. What a hint can add is information about the *future*: that a thread will want the CPU for a long time and should not be switched out every millisecond. The robust way to use that information is to let it change only an efficiency parameter and never priority. Then the classifier's errors, which this study shows are systematic, cost efficiency rather than other programs' responsiveness. Open next steps are real hardware, real user programs, and comparison with the actual Linux scheduler.

References

[1] F. J. Corbató, M. M. Daggett, R. C. Daley. An experimental time-sharing system. *AFIPS Spring Joint Computer Conference*, 1962.

[2] R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.

[3] C. A. Waldspurger, W. E. Weihl. Lottery scheduling: flexible proportional-share resource management. *OSDI*, 1994.

[4] I. Stoica, H. Abdel-Wahab. Earliest eligible virtual deadline first: a flexible and accurate mechanism for proportional share resource allocation. Technical report, Old Dominion University, 1995.

[5] I. Molnar. Completely Fair Scheduler. Linux kernel documentation (`Documentation/scheduler/sched-design-CFS.rst`), 2007.